Goan Konkani Dialect Identification Using Speech Audio

Goa University



MCA(Master of Computer Applications)

Taleigao Plateau, Goa, India-403206

2026-2027

DECLARATION

We, the undersigned hereby declare that this project entitled " Goan Konkani Dialect Identification Using Speech Audio " report has been prepared and submitted by us to Goa University under the guidance of Mr Hanumant Redkar , It has not previously formed the basis for the award of any other in Goa University or elsewhere.

M.C.A

Project Undertaken By:

Roll No	**Name**	**Signature**
2637	Ravi Ramsamuj Chauhan	
2643	Samidha Mote	
2632	Olivia Fernandes	

ACKNOWLEDGMENT

A project like this is never done alone, and we are glad of the chance to thank the people who made it possible. First, we thank our guide, Mr Hanumant Redkar, for patient direction and honest feedback throughout the year. We are equally grateful to the Head of the Department, and to the faculty of the MCA programme at Goa University for giving us access to the lab and computing facilities this work needed.

This project rests on the generosity of the open-source community. We are especially thankful to Reuben CF for publishing the Goan Konkani speech corpus on Hugging Face, which is the raw material of everything in this report, and to Meta AI Research for releasing the MMS-300M speech model for anyone to use. We also thank the linguists whose field studies of Konkani gave us the vocabulary and the map to describe Goa's dialects properly, including the scholars at the Department of Konkani, Goa University. Finally, thank you to our families, friends and classmates, who put up with long nights, half-finished experiments and endless talk about vowels.

INDEX

Sr No	Title	Page No
1	Abstract	6-7
2	Introduction	8-11
3	Literature Review	12-15
4	Existing System	16-17
5	Drawbacks of Existing System	18-19
6	Proposed System	20-21
7	Advantages of Proposed System	22
8	System Requirement	23-24
9	System Design	25-36
10	System Implementation	37-45
11	Live Screenshots	46-48
11	System Limitations	49-50
12	Future Scope	51-52
13	Conclusion	53-54
14	Bibliography	55-56

ABSTRACT

Konkani is the official language of Goa, but anyone who has travelled across the state knows it does not sound the same everywhere. A speaker from Bardez has a lilting, rising rhythm. Someone from Salcete often nasalises the ends of words. A speaker from the Sattari foothills talks quickly and drops short vowels. Native listeners can usually place a speaker within a few sentences, yet computers cannot do this at all. Voice assistants either ignore Konkani or treat it as a flavour of Marathi or Hindi. This project builds a system that listens to a short clip of spoken Konkani and says which of eight regional dialects it most likely belongs to: Antruz, Bardez, Sashti Christian, Sashti Hindu, Canacona, Sattarkar, Pednekari and Kudali. The usual approach is to transcribe the speech to text and then classify the text. For Konkani that approach breaks down, because the language is written in several scripts (Devanagari, Roman, Kannada and Malayalam) with no agreed spelling, and because transcription throws away the very things that separate one dialect from another: pitch, rhythm, vowel length and nasal sound. So this project skips text completely and works directly on the audio.

The system uses Meta's MMS-300M speech model, which had already learned from thousands of hours of speech in over a thousand languages. We added a small classification layer on top and fine-tuned only the top four of its 24 transformer layers. Training data came from 108.5 hours of Goan television broadcasts. A 20-thread script tagged clips by the place names mentioned in their metadata, and the final dataset holds 7,628 cleaned clips.

Each clip passes through a four-step audio cleaning stage before training. Because some dialects have far fewer clips than others, we trained with a class-weighted focal loss so that the model could not win simply by always guessing the biggest class.

The best single model reached 51.18% accuracy and a 46.08% macro F1-score on eight classes, where random guessing would give 12.5%. Combining two models by weighted voting raised accuracy to 53.40%, and the correct dialect was among the top two guesses 84.10% of the time. Most mistakes were between dialects that are neighbours on the map, which matches how dialects actually blend into one another. A layer-depth experiment showed that unfreezing eight layers instead of four made results worse, because the model started recognising the television presenters' voices rather than their dialects. On an Apple M4 laptop, a six-second clip is processed in about 48 milliseconds, and the whole system runs inside a simple web page that accepts a microphone recording or an uploaded file.

INTRODUCTION

Background

Language technology has moved quickly over the last decade. Phones transcribe our speech, translate signs through the camera and answer questions out loud. But almost all of this progress has been built around a small number of languages with huge amounts of recorded and written material. For the hundreds of Indian languages and dialects outside that group, the tools are either weak or missing.

Spoken language identification is one of the basic building blocks of speech technology.

Before a system can transcribe or translate anything, it often needs to know what is being spoken. Dialect identification is the harder cousin of that task. Telling Hindi from Tamil is easy, since the sounds, words and grammar are completely different. Telling one regional variety of Konkani from another is much harder, because speakers share almost the same words and grammar. The differences sit in how the words are pronounced, how long the vowels are held, how the voice rises and falls at the end of a question and how much air passes through the nose.

Konkani and Its Dialects

Konkani belongs to the Indo-Aryan family and is spoken along the western coast of India, mainly in Goa, coastal Karnataka, parts of Maharashtra and Kerala. It is listed in the Eighth Schedule of the Indian Constitution and is the sole official language of Goa. Unlike Hindi or Marathi, Konkani never had a single dominant standard form. Instead

it behaves like a chain of neighbouring varieties, where the way people speak can change noticeably within fifteen or twenty kilometres.

The Eight Regional & Sociocultural Dialects

For the purposes of this investigation, we establish an eight-class dialect taxonomy based on the seminal field dialectology surveys of Dr. Shenoi Goembab, Dr. S.M. Katre (1966), Dr. A.M. Ghatage (1971), and modern phonetic classifications by the Department of Konkani at Goa University. These eight dialects represent the core regional and sociocultural varieties spoken across Goa and its border regions:

1. Antruz (Central Standard / Ponda Taluka): Characterized by balanced vowel durations, moderate pitch modulation, conservative Sanskrit-derived phonology, and serving as the informal basis for modern written Devanagari Konkani literature.

2. Bardez / Bardeskari (North Coastal / Bardez Taluka): Noted for distinct high-pitched melodic intonation, sing-song cadence, vowel lengthening in sentence-final position, and historical Catholic register markers influenced by early Franciscan missionary interactions.

3. Saxtti / Sashti Christian (South Coastal / Salcete Taluka): Characterized by heavy nasalization of terminal vowels (~a, ~e, ~o), widespread Portuguese loanword incorporation (religio-legal and household terms), and relaxed intervocalic retroflex consonant articulation.

4. Sashti Hindu (South Central / Salcete Interior & Quepem): Retains the grammatical framework of Salcete Konkani but exhibits dental plosive clarity, absence of extreme nasalization, and classical Indo-Aryan lexical roots.

5. Canacona (Far South / Canacona Taluka): Features retroflex lateral approximants, distinct vowel shifts towards Kannada phonetics, and unique Dravidian-influenced postpositions resulting from proximity to coastal Karwar.

6. Sattarkar (North-Eastern Ghats / Sattari Taluka): A rugged, rapid-tempo dialect spoken in the Sahyadri foothills, characterized by schwa-deletion, abrupt glottal stops, and lexical preservation of pre-colonial agrarian terminology.

7. Pednekari (Far North / Pernem Taluka): Exhibiting strong phonetic continuum blending with Southern Malvani, characterized by open-vowel rounding and unique verbal inflections distinct from Central Antruz.

8. Kudali (North-Eastern Border / Sindhudurg continuum): Spoken across the Goa-Maharashtra frontier, featuring aspirated stops, contracted auxiliary verbs, and archaic Konkani verbal paradigms.

Dialect	Where	Community	What it sounds like
Antruz	Ponda, interior Tiswadi	Mostly Hindu	Even vowel length, neutral intonation; closest to the written Devanagari standard
Bardez	Mapusa, Calangute, Aldona	Catholic and Hindu	Sing-song rhythm, high pitch peaks, long final vowels
Sashti Christian	Margao, Velim, Salcete coast	Mostly Catholic	Strong nasal vowels, Portuguese loanwords, softer retroflex sounds
Sashti Hindu	Salcete interior, Quepem	Mostly Hindu	Clear dental sounds, little nasalisation, conservative vowels
Canacona	Canacona, Karwar border	Mixed, farming	Lowest pitch, Kannada-influenced sounds, retroflex lateral
Sattarkar	Sattari (Valpoi, Keri)	Forest and farming communities	Fast tempo, dropped schwas, abrupt glottal stops
Pednekari	Pernem (Arambol, Dhargal)	Farming and fishing	Rounded open vowels, shares sounds with Malvani
Kudali	Bicholim, Sawantwadi border	Border trade, farming	Aspirated stops, older verb forms

Why this project matters?

Konkani is under pressure. Rapid urbanisation, migration into Goa's coastal towns, and the dominance of English and Hindi in school, work and media mean that younger people in cities often code-switch and lose the sound patterns of their grandparents' villages. At the same time, voice assistants such as Google Assistant, Siri and Alexa do not properly support Konkani; they either ignore it or mistake it for Marathi or Hindi. A tool that can hear dialects is useful in several practical ways:

- Preservation: sorting and cataloguing archives of old radio and village recordings by dialect, which today is done by hand, if at all.
- Linguistic research: giving dialectologists a quick, repeatable way to check how a recording compares with known regional patterns.
- Better voice technology: a speech assistant that first recognises the dialect can then pick the right speech-to-text model for it.
- Public services: emergency or government announcements can be delivered in a dialect people actually recognise as their own .

LITERATURE REVIEW

Language identification versus dialect identification:

Spoken language identification (LID) decides which language is being spoken, for example Hindi or English. Dialect identification (DID) is the harder cousin: it must separate varieties of one language that share most of their words and grammar. The differences lie in small things, such as how a vowel is shaped, how the pitch rises at the end of a question, and how long a syllable lasts. Two people can say the same sentence and differ only in melody.

How the field has moved over time:

Statistical models (1990s – 2000s)

The first practical systems described speech with MFCCs, a compact summary of the sound spectrum, and modelled each language with Gaussian Mixture Models, often adapted from a Universal Background Model. This worked for telling apart distant languages but did badly on dialects. These models treat speech as a bag of independent frames and cannot easily capture longer patterns like intonation over a whole sentence.

i-vectors (2010s)

Dehak and colleagues introduced i-vectors, which squeeze a whole recording into one short vector that captures both speaker and channel characteristics . Paired with a classifier called PLDA, i-vectors were the standard for nearly a decade. They improved accuracy, but they still treated speech as an unordered collection of frames and missed rhythm and melody.

x-vectors and deeper networks (2018 onwards)

Snyder et al. replaced i-vectors with x-vectors, produced by a time-delay neural network that pools information across the whole utterance . Later models such as ECAPA-TDNN added attention over channels and combined features from several layers . These systems beat i-vectors on international language recognition tests, but they needed large amounts of labelled speech and usually started from hand-built spectrogram features, which discards fine timing and phase detail.

Self-supervised speech transformers (2020 onwards) The biggest change came with wav2vec 2.0 . Instead of needing transcripts, this model learns from audio alone: parts of the signal are hidden and the network must work out what belongs there. After this "pre-training" on thousands of hours of unlabelled speech, the model can be fine-tuned for a task with surprisingly little labelled data. Transformers sit at the heart of this idea. Their self-attention mechanism lets every moment of the recording look at every other moment, so a nasal vowel at the start of a sentence can be linked to the pitch rise at the end.

Meta's Massively Multilingual Speech (MMS) project extended this to over 1,400 languages using more than 50,000 hours of speech . The MMS-300M model has a 7-layer convolutional front end that turns raw audio into a stream of 50 feature vectors per second, followed by 24 transformer layers and about 317 million parameters. Its training material included Konkani .

Dealing with unequal class sizes

In any real collection of recordings, some dialects are far more common than others. A standard cross-entropy loss then lets the large classes dominate: a model can score well just by guessing the big ones. Lin et al. proposed focal loss for image detection. It multiplies the usual loss by a factor $(1 - p_t)^{\gamma}$ that shrinks the loss on examples the model already gets right, leaving training effort for the difficult ones. We borrow this idea for speech and add a per-class weight so that rare dialects count for more.

Fine-tuning large models:

When a big pre-trained model is adapted to a small dataset, a key decision is how many layers to update. Updating all of them risks "catastrophic forgetting", where the model loses useful general knowledge, while updating too few leaves it unable to adapt. Research on speech models suggests that lower layers capture raw acoustics, middle layers capture a lot of speaker information, and the top layers capture more abstract content such as phonetics and prosody.

Data augmentation and optimisation:

Park et al. showed that randomly hiding bands of time or frequency during training (SpecAugment) makes speech models more robust . For optimisation, AdamW with decoupled weight decay has become the standard choice for fine-tuning transformers, usually paired with a short warm-up and a smooth decay of the learning rate. We use all of these.

Gap this project fills:

Putting the pieces together, we found no earlier work that

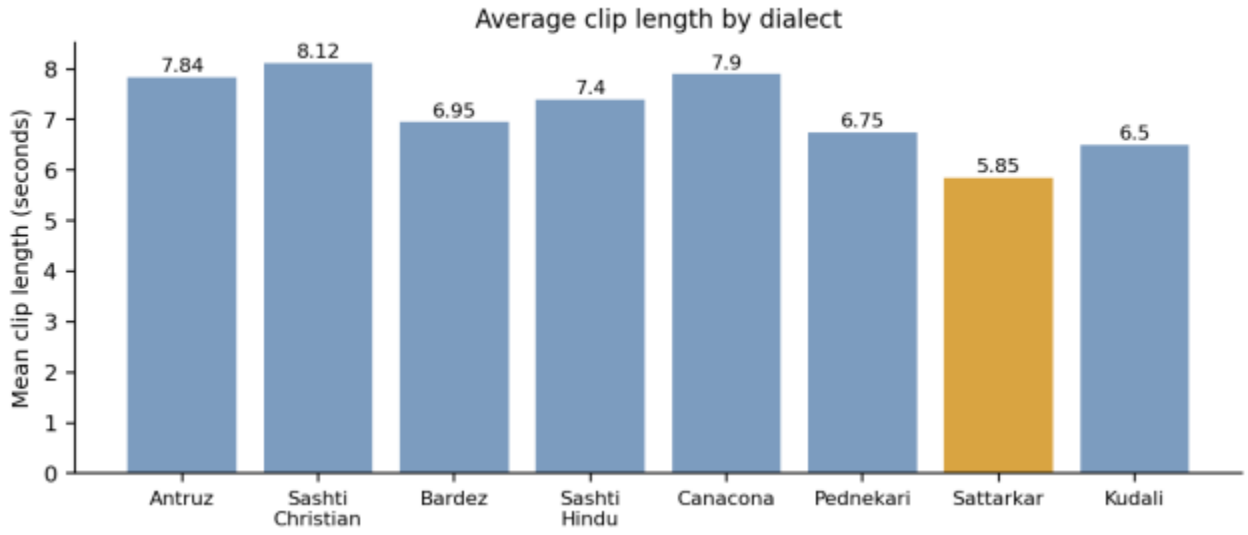
(a) builds a labelled, multi-channel dialect dataset for Goan Konkani,

(b) classifies eight regional dialects directly from audio, and

(c) studies how many layers of a speech transformer should be unfrozen to avoid memorising speakers. This project does all three .

EXISTING SYSTEM

Most systems that try to recognise an accent or dialect from speech work in two steps. In step one, an automatic speech recognition (ASR) engine such as Google Speech-to-Text, Whisper or Kaldi writes down what was said. In step two, a text classifier, for example a TF-IDF model or a fine-tuned BERT, reads that text and guesses the dialect from word choice and grammar. For some languages this works nicely. British and American English differ in words like “lift” and “elevator”, “flat” and “apartment”, so a text classifier can pick up on them. The method assumes that dialects differ mainly in vocabulary, and that the ASR step is accurate.



(Average clip length differs by dialect: Sattarkar speakers are fastest, Sashti Christian speakers the slowest. Pace is another clue that text-only methods lose.)

Existing speech-based methods :

Apart from the two-step approach there are also purely acoustic methods. Classical systems use MFCC features with GMMs or i-vectors; newer ones use x-vectors or ECAPA-TDNN

embeddings . These do work on sound, but they were designed and tested on languages with large labelled datasets. For Konkani dialects there is no such dataset, and these models cannot take advantage of the huge amount of unlabelled speech that self-supervised models learn from.

Existing digital tools for Konkani :
At the time of writing, we are not aware of a public tool that identifies Goan Konkani dialects. Mainstream voice assistants handle Konkani poorly or not at all. Academic corpora are small, read-aloud, or restricted. As a result, dialect identification in Goa today is mostly done by human listeners, which is slow, subjective and does not scale to large archives.

A summary of how existing approaches compare

Approach	Input	Main strength	Main weakness for Konkani
ASR then text classifier	Written text	Easy to build for big languages	Script confusion, spelling chaos, ASR errors, prosody is lost
MFCC + GMM	Hand-built spectral features	Light and fast	Cannot model sentence-level rhythm and melody
i-vector + PLDA	Compact utterance vector	Strong for speakers and languages	Ignores order of events in speech; needs lots of labelled data
x-vector / ECAPA-TDNN	Learned embedding from spectrograms	Good accuracy where data is plentiful	Needs large labelled corpora, which Konkani lacks
Human listeners	Live or recorded audio	Very accurate for natives	Slow, subjective, not scalable

DRAWBACKS OF EXISTING SYSTEM

The two-step approach looks sensible on paper, but when we examine it closely for Konkani it breaks down in four separate places.

One language, four scripts

Konkani is written in several scripts, and which one a person uses often depends on their community and region:

• Devanagari is the script officially recognised under the Goa Official Language Act of 1987, and is used mostly by the Hindu majority and by government offices.

• Roman (Romi) has a literary history of roughly 450 years. It is used by the Catholic community, in church services, tiatr plays and periodicals.

• Kannada script is used by hundreds of thousands of Konkani speakers in coastal Karnataka.

• Malayalam script is used by the Gowda Saraswat community in Kochi. An ASR system has to choose one. The same question, “How are you?”, appears as Koxem asa? in Roman and as कशेआसा? in Devanagari. To a text model those are completely unrelated strings. Whichever script we choose, we lose or bias a large part of the speaking community.

Spelling is not standardised:

Even within Roman script, people spell things as they like. “How are you?” is written Koso asa, Koxem asa, Kosso asa, Coxem assa and Koxe asa in everyday messages. Nasal vowels like ã, õ and ẽ are hard to type on a phone keyboard, so people simply

skip them. A text classifier trained on such noisy spellings learns the habits of typists more than the habits of dialects.

Writing it down erases the dialect:

This is the most serious drawback. What makes Bardez sound like Bardez is largely prosody: a lifted pitch, a drawn-out last vowel, a sing-song rhythm. What makes Sashti Christian speech recognisable is its nasal resonance. None of that survives transcription. If a Bardez speaker and an Antruz speaker say the same sentence with the same words, the ASR engine hands the classifier identical text. By the time step two begins, the dialect signal has already been deleted in step one.

Errors pile up

In low-resource languages, word error rates on casual, rural speech are often above 45%. When that noisy text is passed on to a classifier, mistakes compound. We also see "mode collapse": the classifier falls back on predicting the most common dialect (Antruz in our case) for almost everything, ignoring the smaller dialects such as Canacona, Sattarkar and Kudali.

Other practical drawbacks

Beyond these four main problems, existing approaches carry practical costs. They generally need large labelled corpora that Konkani does not have. They usually lack any explanation of why a result was given, which makes them hard for linguists to trust. And most are built for cloud servers, which makes them awkward to run on a local machine in a classroom or a field-recording setting.

PROPOSED SYSTEM

The core idea:

Our proposal is simple to state. Instead of turning speech into words and then guessing the dialect from the words, we let a neural network listen to the audio itself. The network sees the real shape of the sound wave, including the pitch movements, vowel lengths, nasal resonance and pace of speech, and it decides on the dialect from that. We call this the acoustic-first approach. Because no text is involved, the system does not care whether the speaker would write in Devanagari, Roman or Kannada script. It also does not care how anyone spells Koxem asa. What matters is the sound of the voice.

Overview of the pipeline

The system has five connected stages”

1. **Audio cleaning**. Recordings are converted to 16 kHz mono and passed through a four-step filter chain.
2. **Convolutional feature encoder**. Seven small convolution layers turn the waveform into a sequence of feature vectors, one every 20 milliseconds.
3. **Transformer backbone.** 24 self-attention layers (from Meta’s MMS-300M) build a context-aware description of the speech. The bottom 20 layers stay frozen; only the top 4 are fine-tuned.
4. **Pooling and classification head**. A masked average over time produces one summary vector for the clip, and a small custom network turns it into eight scores.
5. **Focal loss training**. A cost-sensitive loss makes sure rare dialects are learned as seriously as common ones.

Prediction on long recordings

Transformers use memory that grows with the square of the input length, so a one-minute recording cannot be fed in at once. At prediction time we cut long audio into 6-second windows with a 3.5-second step (so consecutive windows overlap by 2.5 s), classify each window, and average the probabilities. This also smooths out a ringing phone or a cough in one window. The final dialect is the one with the highest average probability

Two models are better than one

We trained several versions of the model. Version 5.0 is the most precise on the earlier, smaller dataset; version 6.0 was trained on the full Master Dataset and generalises across channels. Taking a weighted average of their probabilities (the dual-engine ensemble) gives better accuracy than either alone. The system also compares the two predictions: if they pick the same dialect it shows a High-confidence consensus badge; if they disagree it shows an Ambiguous regional boundary warning, which is an honest signal that the speaker may sit between two dialect areas.

A bare label such as “Bardez, 68%” is not very useful to a linguist. The system also produces a short reasoning card: the acoustic features typical of the predicted dialect, the margin between the winner and the runner-up, and whether both models agreed. This does not claim to reveal exactly what the network used, but it helps a human check the answer against known dialect features .

ADVANTAGES OF PROPOSED SYSTEM

- It does not depend on any writing system: The model works equally well whether the speaker's community uses Devanagari, Roman or Kannada script, and it sidesteps the long-running debate over which script Konkani should use. This is not just a convenience; it keeps the technology neutral and usable for everyone.

- It keeps the clues that define dialects: Pitch contour, vowel length, nasalisation and speaking pace all stay in the signal.

- Fewer things can go wrong :With no speech-to-text step, there is no ASR error to carry forward. The whole system is trained as one piece, from waveform to dialect, so every part is optimised for the same goal.

- Fair treatment of small dialects :Focal loss with inverse-frequency weights keeps rare dialects such as Kudali, Sattarkar and Pednekari from being overlooked.

- It builds on existing knowledge instead of starting from scratch.

- Easy to extend :The same pipeline can be retrained for other dialect sets, other languages, or other speech categories (for example, age group or emotion) by changing the labels and the number of outputs

FUNCTIONAL REQUIREMENT

- Accept an audio file (WAV, MP3, M4A, FLAC, AAC or OGG) by upload or drag-and-drop.
- Record speech live from a microphone at 16 kHz, 16-bit mono.
- Reject recordings shorter than 1 second and split recordings longer than 15 seconds into 6-second windows.
- Clean every recording with the same four-stage filter used in training.
- Run one or both models and combine their probabilities.
- Display the predicted dialect, a probability for all eight dialects, a confidence/agreement badge and a short reasoning card.
- Let the user pick which model(s) to use: v5.0, v6.0, the dual ensemble, or the tri-engine ensemble.
- Allow batch use from a command-line client and shell scripts. • Store clip metadata and prediction logs in a consistent structure.

Non-functional requirements

- Speed: under 100 ms for a 6-second clip on the target laptop GPU.
- Reliability: the interface must stay responsive while the model is running (all server calls are asynchronous).
- Reproducibility: fixed random seed (42), pinned library versions, and a stored train/test manifest.
- Portability: must run on macOS (Apple Silicon), Linux (CUDA) and fall back to CPU.

- Usability: a person with no technical background should be able to drop in a file and read the result.
- Honesty: predictions must come with probabilities rather than a bare label.

HARDWARE AND SOFTWARE REQUIREMENTS

Component	Specification	Role
GPU	NVIDIA Tesla T4 (16 GB) / A100 (40 GB)	Model training
Tensor cores	320 (T4) / 432 (A100)	FP16 mixed-precision maths
CPU	Intel Xeon E5-2699 v4 / AMD EPYC 7B12	Data loading
RAM	12.7 GB (standard) / 51 GB (high-RAM)	Caching clips and manifests
OS	Ubuntu 22.04 LTS	Runtime
CUDA / cuDNN	12.2 / 8.9	GPU acceleration

Tool	Version	What we used it for
PyTorch	2.5.1	Model, training loop, mixed precision
TorchAudio	2.5.1	Loading audio, resampling
Transformers (Hugging Face)	4.49.0	MMS-300M backbone and processor
FFmpeg	7.1.0	Four-stage audio filtering
Librosa	0.10.2	Pitch, formants, spectrogram analysis
SoundFile	0.13.1	Fast audio file I/O
Scikit-learn	1.6.1	Stratified split, precision / recall / F1
FastAPI + Uvicorn	0.115 / 0.32	Model server on port 8000
Gradio	4.44.1	Web dashboard with microphone input
Java (JDK)	17	Desktop client built with Swing
Graphviz / PlantUML	12.2 / 1.2024	Diagram generation

SYSTEM ANALYSIS & DESIGN

Brainstorming

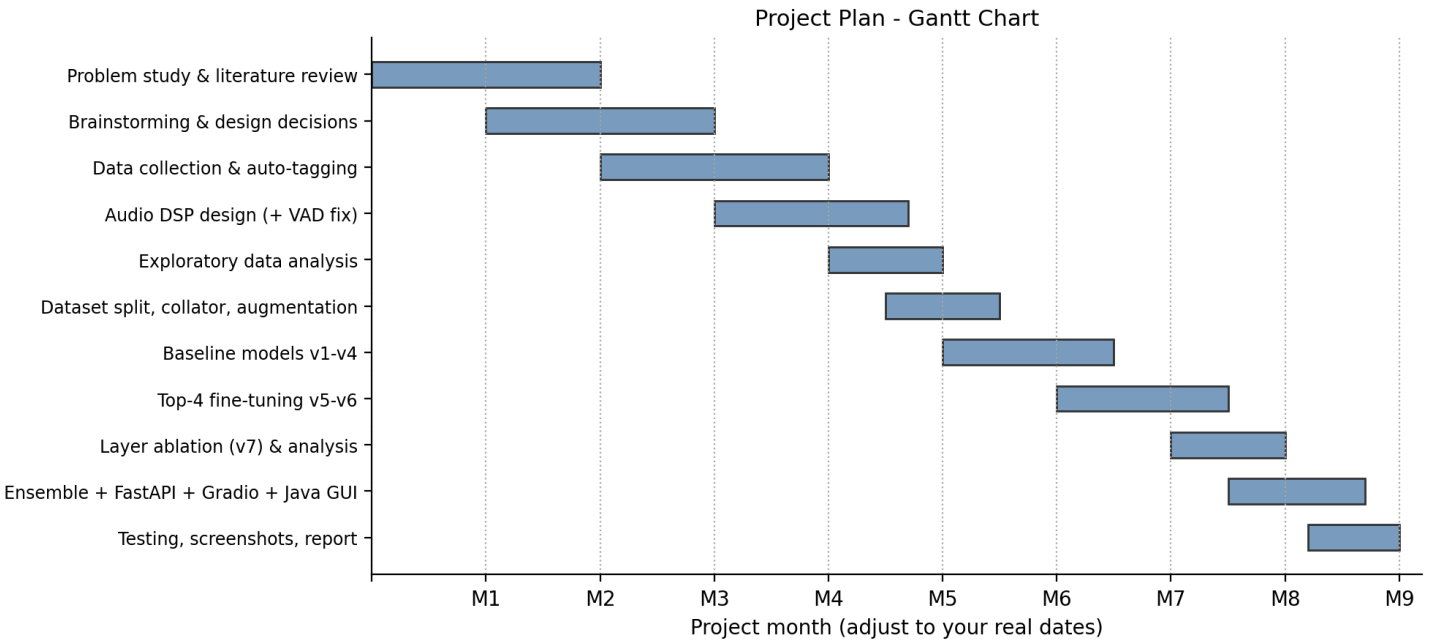
We began with one question: how can a computer tell a Bardez speaker from a Sattari speaker? Three directions came up straight away: (a) transcribe the speech and classify the text, (b) use classic audio features such as MFCC with a standard classifier, and (c) feed the raw audio to a large pre-trained speech model. Each was weighed against what we knew about Konkani: many scripts, no standard spelling, very little labelled data, and dialects that differ mainly in melody**.**



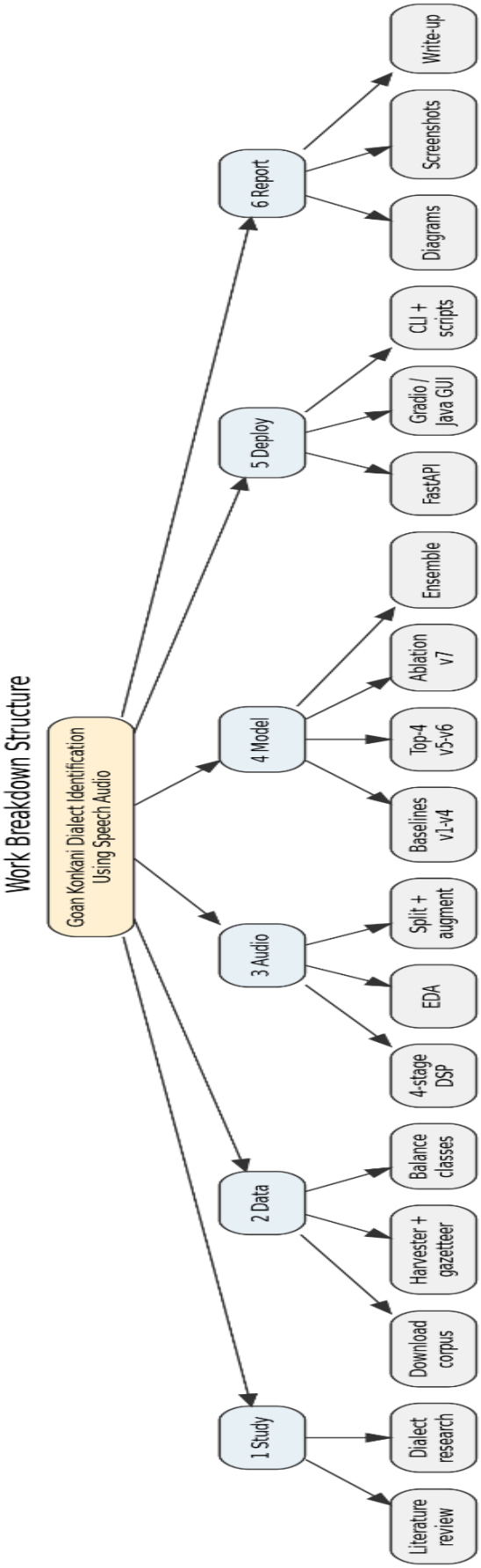
Problems met and fixed during development

Problem	Cause	Fix
Model collapsed onto Antruz (v1, v2)	Cross-entropy with imbalanced data	Focal loss, balanced dataset
Accuracy fell on Bardez and Sashti Christian	End-silence trimming cut final vowels	Removed stop_periods; leading only
RuntimeError: tensor a (21) vs b (25)	Mask length rounding in batches	Custom collator, pad to multiples of 320
AssertionError: can only test a child process	DataLoader workers on Python 3.13	Persistent workers, forkserver, or one worker
8-layer model got worse after epoch 8	Memorised speakers	Top-4 unfreezing only

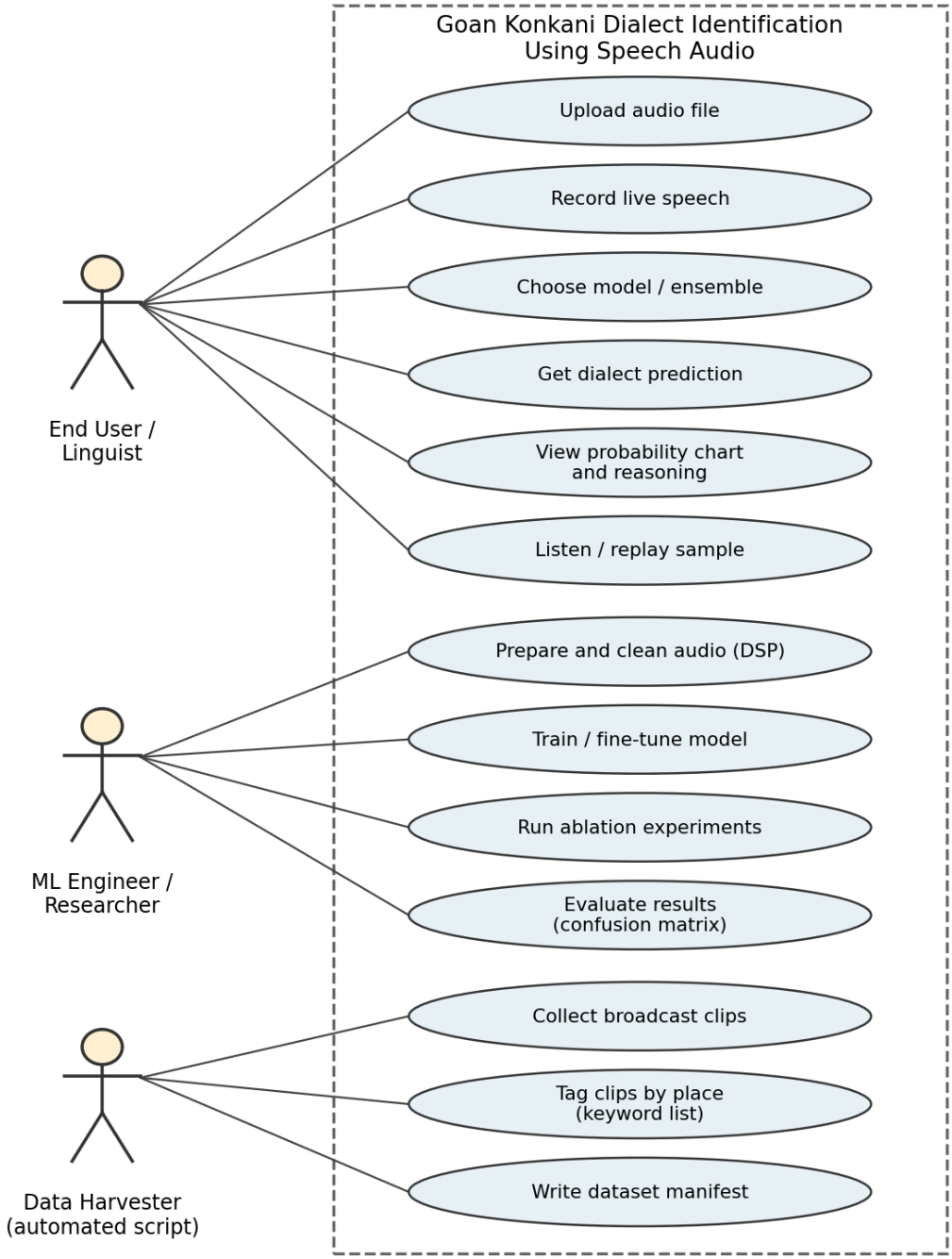
Project Plan



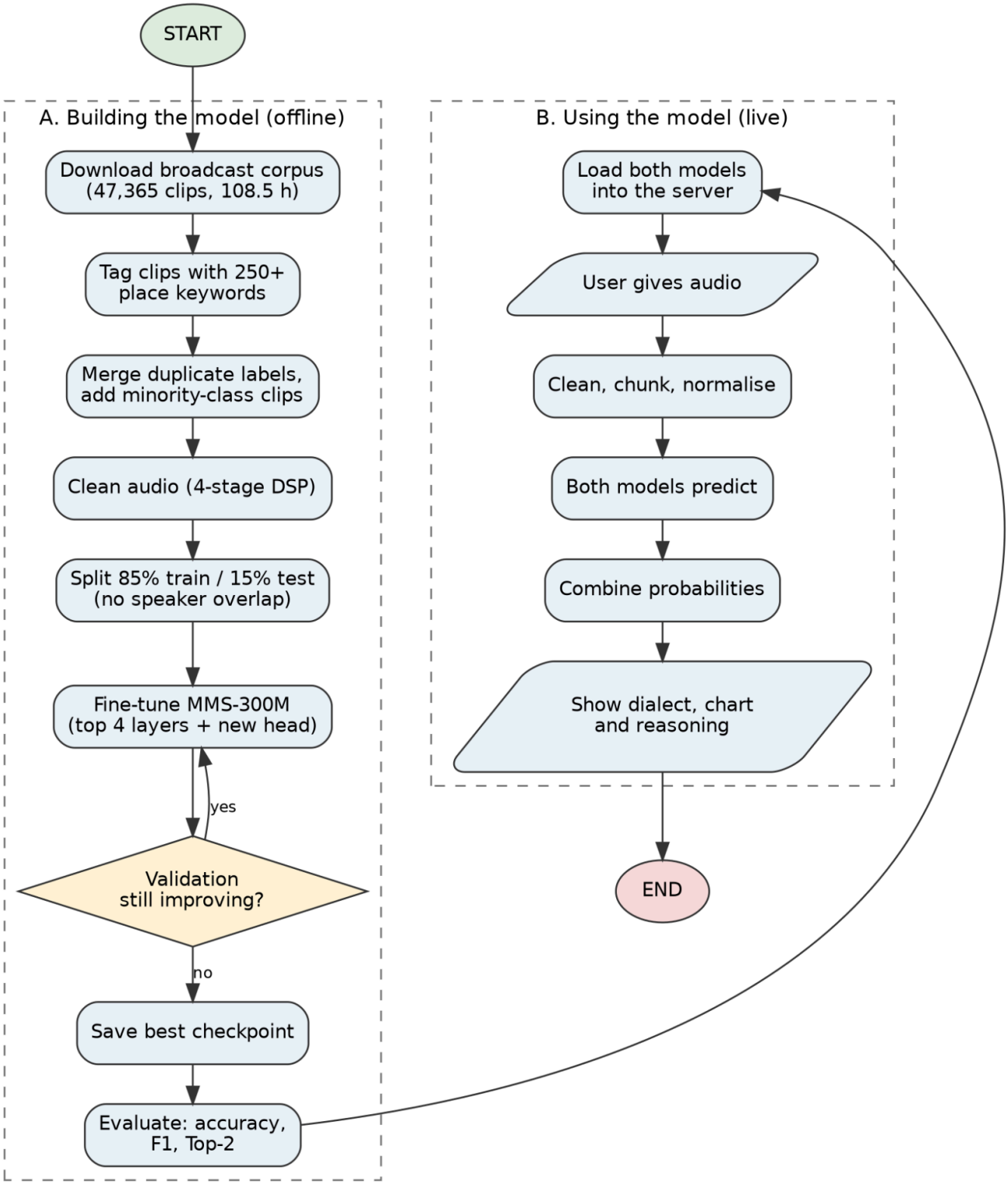
Work Breakdown



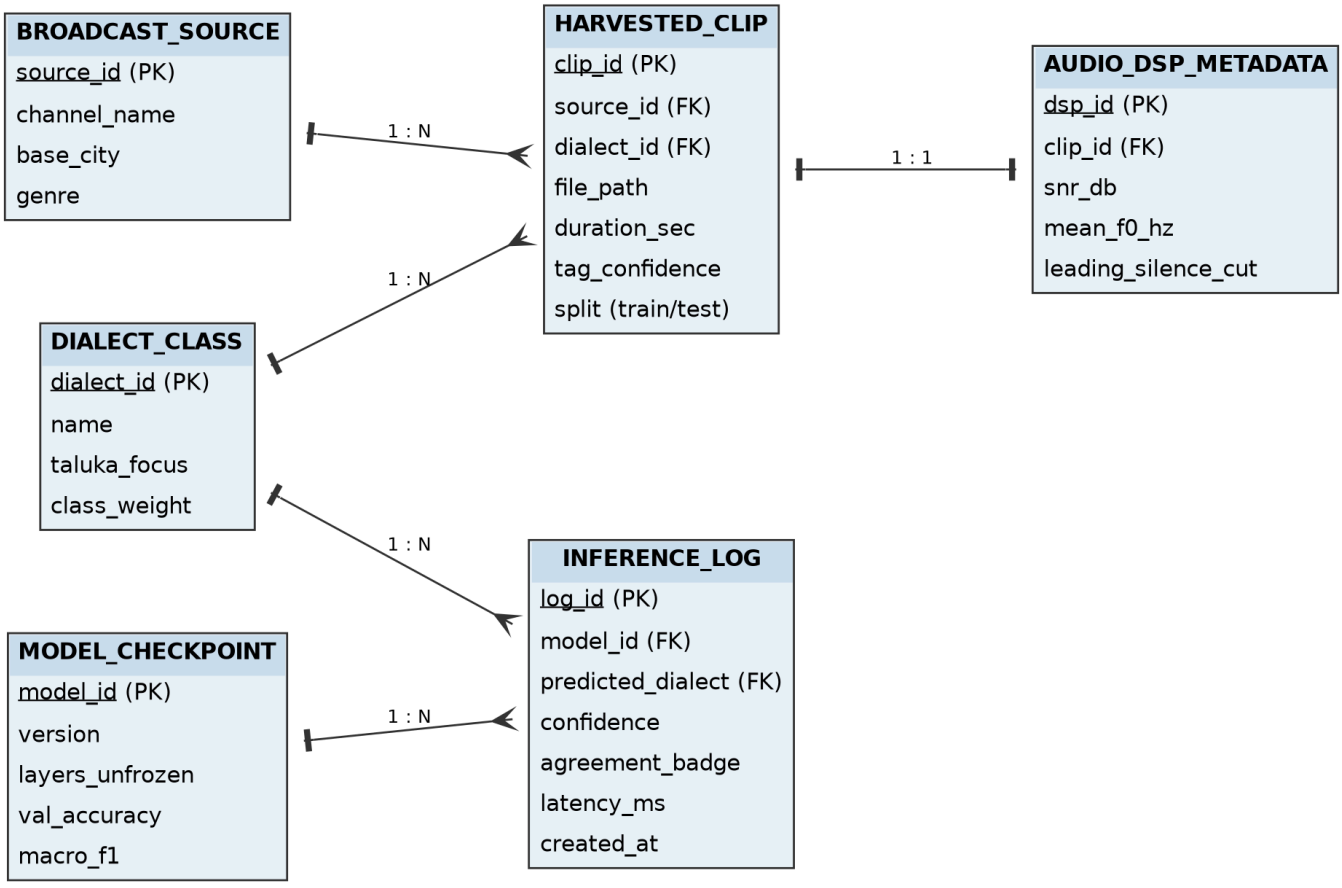
USE CASE DIAGRAM



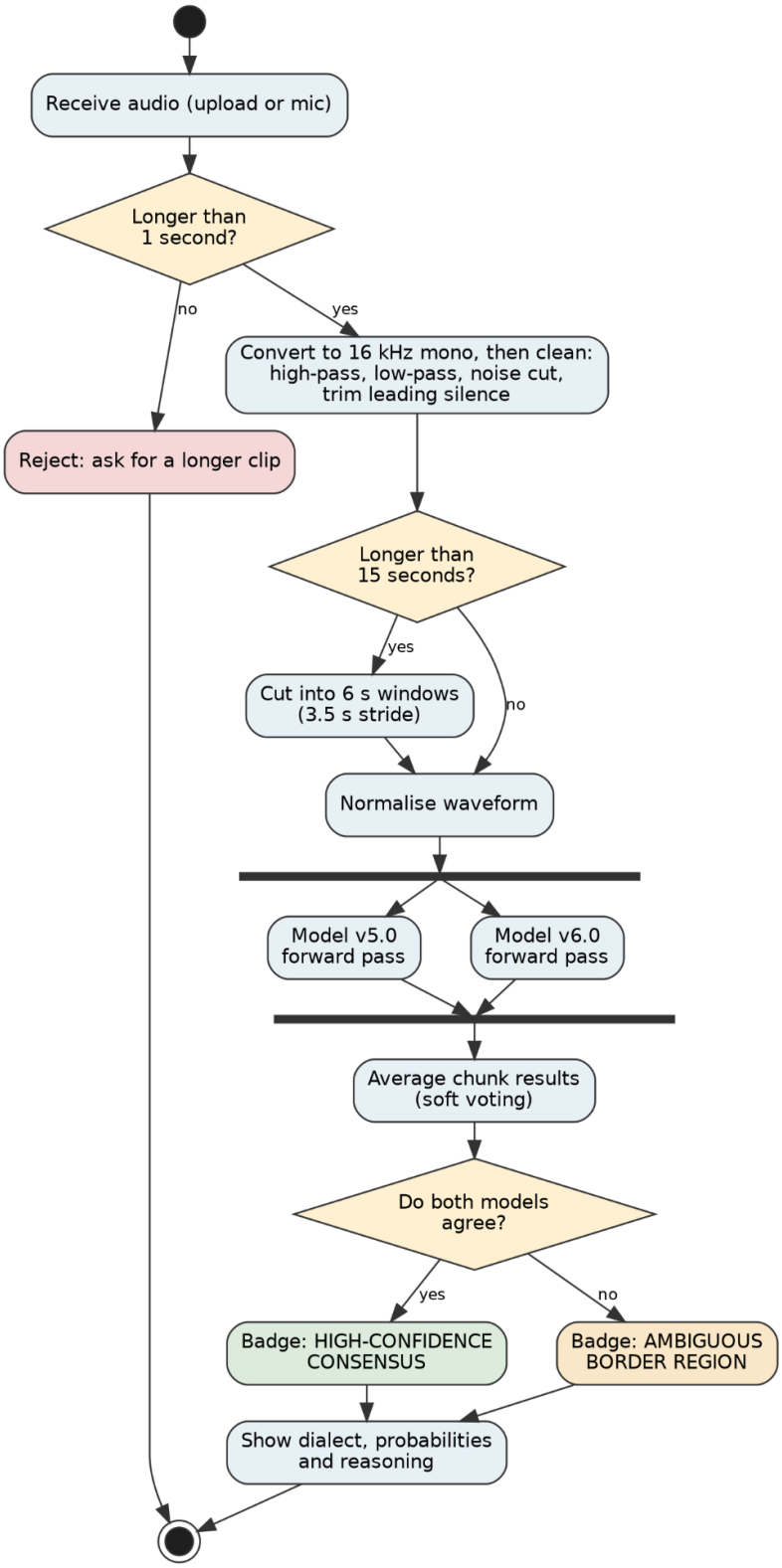
FLOW CHART


START
A. Building the model (offline)
Download broadcast corpus (47,365 clips, 108.5 h)
Tag clips with 250+ place keywords
Merge duplicate labels, add minority-class clips
Clean audio (4-stage DSP)
Split 85% train / 15% test (no speaker overlap)
Fine-tune MMS-300M (top 4 layers + new head)
yes
Validation still improving?
no
Save best checkpoint
Evaluate: accuracy, F1, Top-2
B. Using the model (live)
Load both models into the server
User gives audio
Clean, chunk, normalise
Both models predict
Combine probabilities
Show dialect, chart and reasoning
END

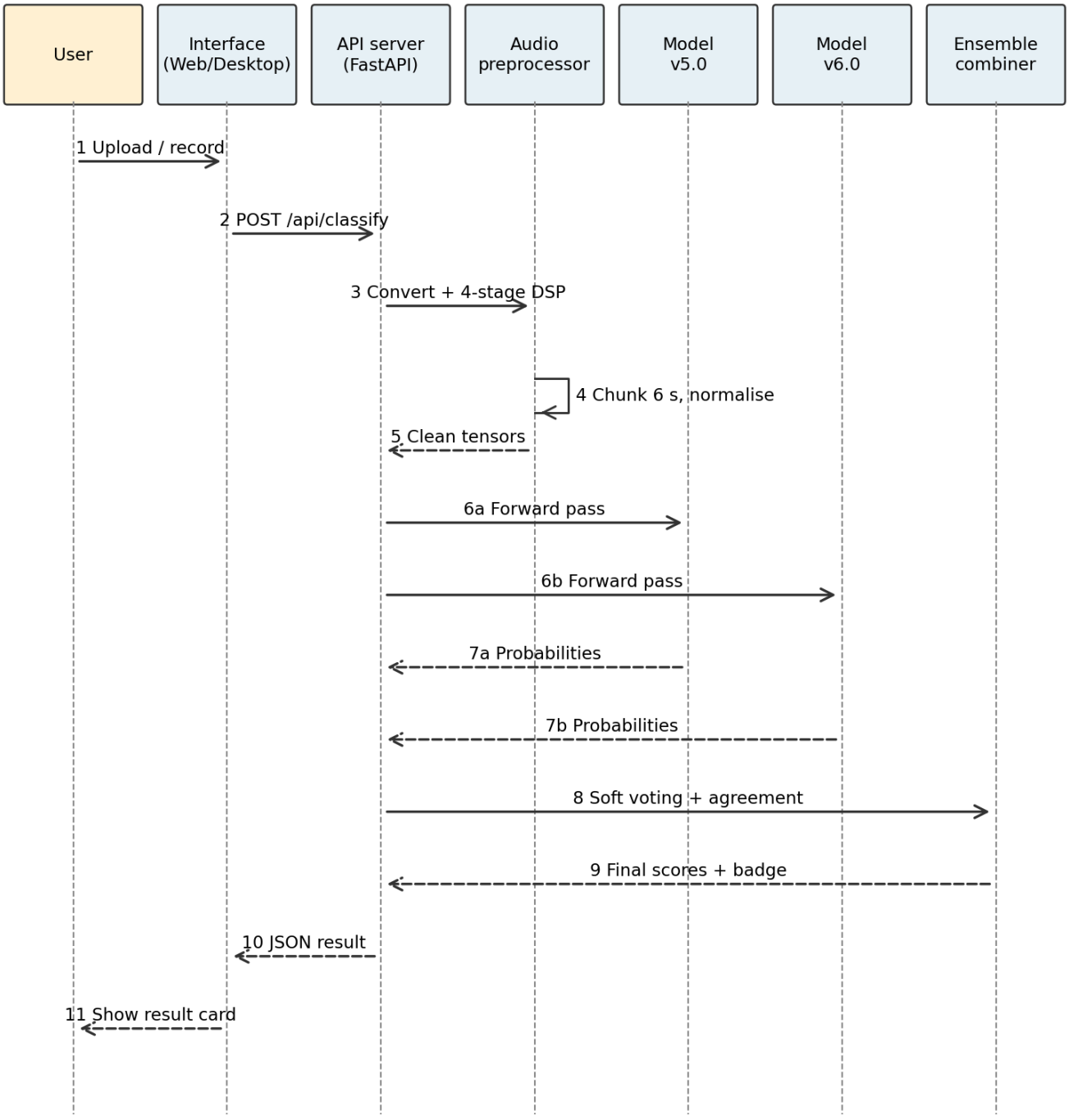
ERD DIAGRAM


BROADCAST_SOURCE
source_id (PK)
channel_name
base_city
genre
HARVESTED_CLIP
clip_id (PK)
source_id (FK)
dialect_id (FK)
file_path
duration_sec
tag_confidence
split (train/test)
AUDIO_DSP_METADATA
dsp_id (PK)
clip_id (FK)
snr_db
mean_f0_hz
leading_silence_cut
DIALECT_CLASS
dialect_id (PK)
name
taluka_focus
class_weight
INFERENCE_LOG
log_id (PK)
model_id (FK)
predicted_dialect (FK)
confidence
agreement_badge
latency_ms
created_at
MODEL_CHECKPOINT
model_id (PK)
version
layers_unfrozen
val_accuracy
macro_f1
1 : N
1 : 1
1 : N
1 : N
1 : N

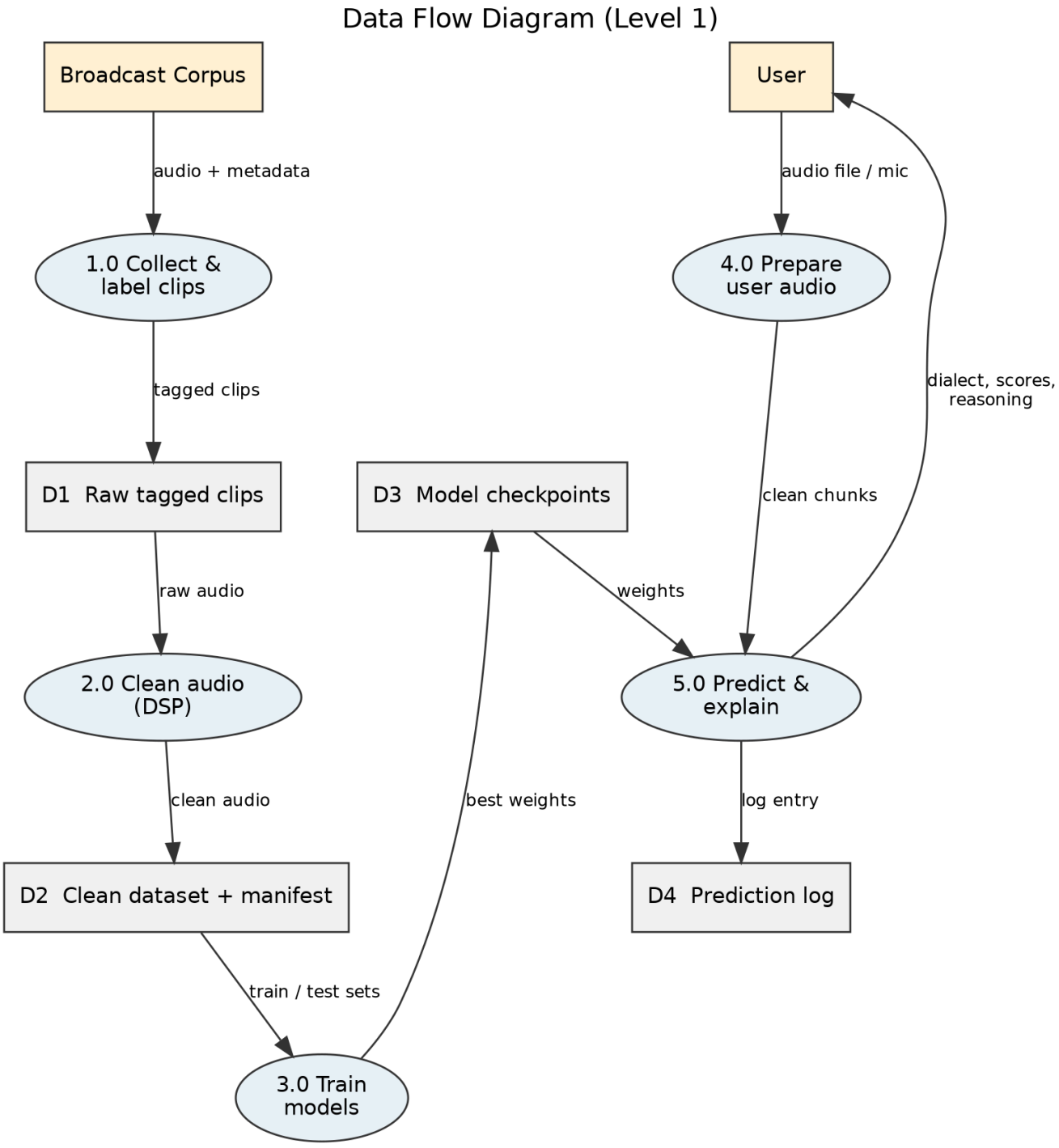
ACTIVITY DIAGRAM



SEQUENCE DIAGRAM

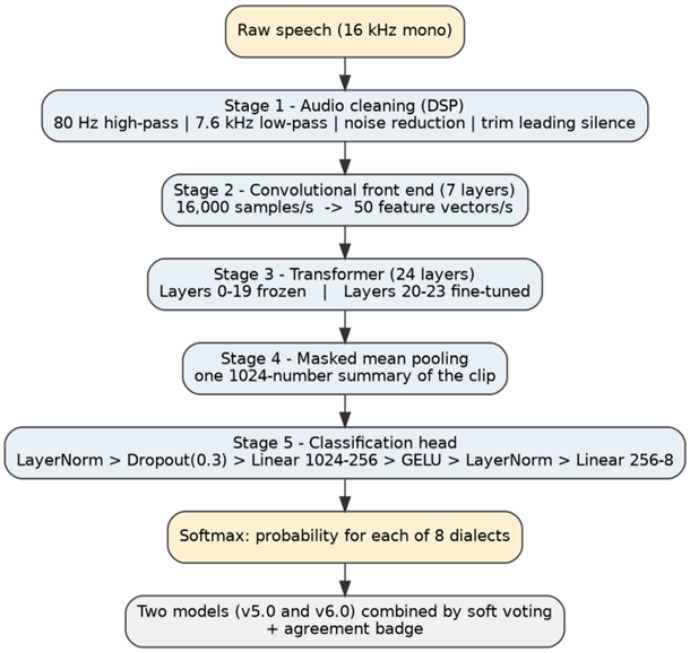


DATA FLOW DIAGRAM



Overall architecture

Figure 6.1 shows the full path from raw audio to dialect probabilities. The recording is cleaned, passed through a convolutional front end and then through the transformer layers of MMS-300M. The result is summarised into one vector and turned into eight probabilities by a small classification head. Two such models vote together for the final answer.



Stage 2 in detail: from waveform to feature vectors

One second of 16 kHz audio is 16,000 numbers. That is far too much detail for a transformer to attend over directly. The convolutional encoder reduces this by a factor of 320, so that each second of speech becomes 50 feature vectors of size 512, one for every 20 ms. Each of the seven blocks has a one-dimensional convolution, a normalisation step and a GELU activation.

Stage 3: the transformer

The feature vectors are projected up to 1,024 dimensions and given positional information, then passed through 24 transformer blocks. Each block has 16-head self-attention and a feed-forward network with an inner size of 4,096. The attention operation is

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_k}) V$$

In everyday terms, every moment of the recording asks "which other moments are relevant to me?" and mixes in information from them. This is exactly what dialect recognition needs: a nasal vowel in the first syllable may only make sense in relation to a rising pitch eight syllables later. The weights of this transformer already encode a lot of knowledge about human speech from more than 50,000 hours of multilingual audio. We do not want to destroy that, so most of the layers are frozen and only the top few are allowed to change.

Pooling and the classification head

Clips differ in length, so the transformer's output (one 1,024-number vector for every 20 ms) must be summarised into a single vector. We average over time, taking care to ignore padding frames:

$$h = (\Sigma\, m_t \cdot H_t) / (\Sigma\, m_t)$$

where m_t is 1 for a real speech frame and 0 for padding. This vector goes through a small head: LayerNorm, dropout (0.3), a linear layer from 1,024 to 256 units, GELU, another LayerNorm, and a final linear layer from 256 to 8. The eight outputs are turned into probabilities with a softmax.

Focal loss with class weights

Standard cross-entropy loss is L = −log(p_t), where p_t is the probability the model gave to the correct dialect. With unequal class sizes the model can lower this loss simply by favouring the big classes. Focal loss reshapes it:

FL(p_t) = − α_t (1 − p_t)^γ log(p_t), with γ = 2

When the model is already confident and correct (p_t close to 1), the factor $(1 - p_t)^2$ is tiny and the example barely contributes. When the model is wrong or unsure, the factor stays near 1 and the example matters fully. The weight α_t adds a second correction: it is larger for dialects with fewer clips

SYSTEM IMPLEMENTATION

Getting the data, understanding it, cleaning it, training the models, testing ideas, and finally turning the best model into something people can use. Along the way we also describe the problems we ran into, because several of them taught us more than the things that worked the first time.

Step 1:

Getting the data; The source corpus Our raw material is the Goan Konkani Speech corpus published by Reuben CF on Hugging Face. It contains 47,365 clips, about 108.5 hours, recorded from four regional television channels between 2021 and 2023. The clips have no dialect labels. Table 10.1 shows where the audio comes from.

Channel	Base	Typical programmes	Likely dialect lean	Audio
Prudent Media	Panaji	News, panel debates	Central and South (Antruz and Sashti mixed)	38.5 h
Doordarshan News Goa	Altinho, Panaji	Official bulletins, farm programmes	Standard Devanagari (Antruz)	26.0 h
Prime TV Goa	Margao	Civic reports, village interviews, tiatr reviews	South coast and rural talukas	24.5 h
Goa 365	Mapusa, Ponda	Investigative reports, festival documentaries	North Goa and foothills	19.5 h

Labelling 47,000 clips without listening to them Nobody can hand-label tens of thousands of clips. We wrote a multithreaded harvester (20 worker threads) that reads each clip's metadata, such as titles, programme descriptions and place names, and looks for evidence of where the speaker comes from. It uses a gazetteer of more than 250 Goan place names, including villages, talukas, parishes and well-known institutions. If a clip's description mentions Calangute, Anjuna or Aldona, it is tagged

Bardez; mentions of Margao, Velim or Benaulim point to Sashti Christian, and so on. Each rule has a confidence score and only matches scoring 0.90 or higher were kept.

Step 2:

Getting to know the data; Before training anything, we looked closely at the clips. All 7,628 are between 1.0 and 14.95 seconds long, with a mean of 7.41 seconds and a median of 6.75 seconds. The distribution leans slightly towards shorter clips (skewness +0.51). Pace differs across dialects: Sattarkar clips are shortest on average (5.85 s) and Sashti Christian the longest (8.12 s). That fits what native speakers say, that Sattari speech is quick and clipped and Salcete Christian speech is slower, with long, nasal sentence endings. To check that the dialects really are different physically, and not just in our heads, we measured the first three vowel formants (F1, F2, F3, which reflect tongue and mouth shape) and the fundamental frequency F0 (pitch) on 200 random clips per dialect. Table 10.4 shows that the classes do separate.

Step 3:

Cleaning the audio; TV audio is not clean. It has air-conditioning rumble, 50 Hz electrical hum, high-frequency hiss and long silences at the start of clips. If we feed this straight into the network, some of its attention is wasted on modelling noise rather than voices.

- A bug that taught us something about dialects In an early version of step 4 we also removed silence at the end of each clip (the option stop_periods=1). It seemed harmless. But accuracy on Bardez and Sashti Christian fell sharply, by roughly 14 percentage points. When we looked at the waveforms we understood

why. Bardez speakers stretch the final vowel of a sentence into a rising tone, and Salcete Christian speakers finish with a soft nasal murmur. Both are low in energy, so the filter treated them as silence and cut off the last 400 to 800 milliseconds, deleting precisely the sounds that give the dialect away.

Step 4:

Preparing the data for PyTorch Splitting without cheating; We split the 7,628 clips 85% / 15% using stratified sampling (random seed 42), giving 6,483 training clips and 1,145 test clips with the same dialect proportions in each. To make the test honest, clips from the same TV programme or the same speaker were kept entirely in one side. Otherwise the model could score well just by recognising the same newsreader in both sets.

Normalising and augmenting was also needed.

Each waveform is normalised to zero mean and unit variance, $x_norm = (x - mean) / (std + 10^{-7})$, so differences in recording volume do not mislead the model. During training we also apply random augmentations below table shows the content, so the model cannot just memorise the exact recordings.

Data augmentations applied while training

Augmentation	Setting	Why
Time masking	Up to 20 frames (400 ms)	Stops reliance on a single syllable
Frequency masking	Up to 15 channels	Imitates different microphones and phone filters
Time stretching	Speed ×0.95 to ×1.05	Covers natural differences in speaking rate
Gaussian noise	SNR 25 to 35 dB	Makes the model robust to noisy microphones

Training settings

Setting	Value	Reason
Base model	facebook/mms-300m (317 M parameters, 24 layers)	Pre-trained on 1,400+ languages
Optimiser	AdamW, weight decay 0.01	Standard for fine-tuning transformers
Learning rate (head)	3×10^{-5}	New layers must learn quickly
Learning rate (backbone)	1×10^{-5}	Gentle changes to pre-trained layers
Schedule	10% linear warm-up, then cosine decay to 10^{-7}	Stable start, smooth finish
Batch size	16 clips	Fits in 16 GB T4 memory with AMP
Loss	Focal loss, $\gamma = 2$, inverse-frequency weights	Protects small dialects

Two engineering problems we had to solve: A shape-mismatch crash. Midway through development, training stopped with RuntimeError: The size of tensor a (21) must match the size of tensor b (25). The cause was a rounding difference: when batches of different-length clips were padded, the attention mask length computed by the library could differ from the real convolution output by up to four frames.

Crashes with multiple data-loader workers. On Python 3.13 in Colab, using several **DataLoader workers raised AssertionError:** can only test a child process when workers were shut down. Switching to persistent workers with the forkserver start method, or in the worst case a single worker with in-memory caching, fixed it while keeping GPU use above 95%.

Clips shorter than 3 seconds gave poor results (below 28% accuracy) because isolated words such as ghor (house) or udok (water) sound nearly the same across Goa; the difference is in the melody of whole sentences. Very long clips (over 25 seconds) brought in several speakers and ran out of GPU memory. A range of roughly 5 to 15 seconds, with 6-second windows at prediction time, was the best compromise.

Step 5:

Training the model :

Two learning rates; The classification head starts with random weights and needs bigger updates, while the pre-trained transformer layers already hold valuable knowledge. Using a rate three times smaller for the backbone (1e-5 versus 3e-5) lets them adapt gently without "forgetting" what they know.

Class weights; For each dialect c we compute $\alpha c = Ntotal / (K \times Nc)$ and rescale so the average is 1. Antruz, the largest class, gets a weight of 0.584; Kudali, the smallest, gets 2.763, almost five times more. Mixed precision cut GPU memory use from 14.2 GB to 7.4 GB (about 48%) and made training 2.6 times faster.

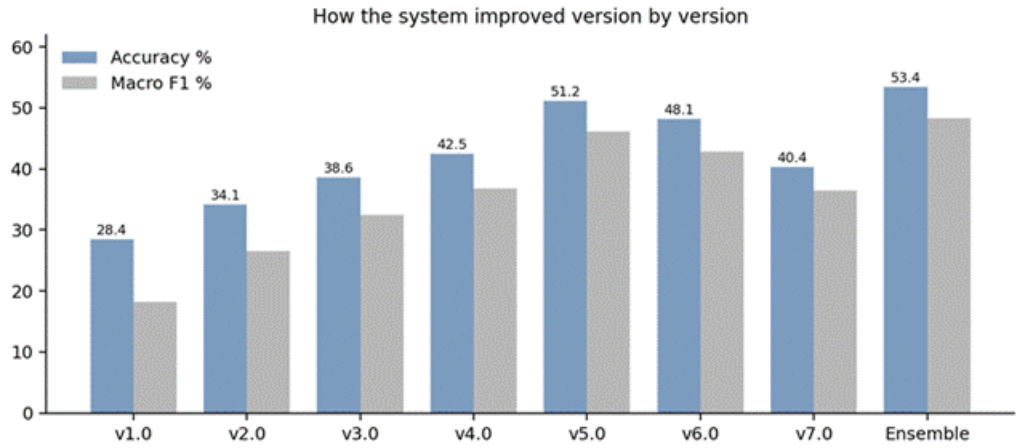
Step 6: Experiments, from version 1 to version 7

The final system is the end point of seven model versions. Each one was a response to something the previous one did badly.

How each model version performed

Version	Clips	Layers trained	Loss	Accuracy	Macro F1	Top-2	What happened
v1.0	1,200	0 (frozen)	Cross-entropy	28.40%	18.20%	48.10%	Collapsed onto Antruz
v2.0	2,800	0	Cross-entropy	34.10%	26.50%	56.40%	Audio filtering removed hum
v3.0	2,800	0	Focal (γ 1.5)	38.60%	32.40%	64.20%	Small dialects recovered
v4.0	5,285	0	Focal (γ 2)	42.50%	36.80%	71.50%	More data, less imbalance
v5.0	5,285	Top 4	Focal (γ 2)	51.18%	46.08%	81.60%	Breakthrough
v6.0	7,628	Top 4	Weighted focal	48.12%	42.80%	82.50%	Full dataset,

							harder test
v7.0	7,628	Top 8	Weighted focal	40.35%	36.39%	74.80%	Overfitted
Ensemble	7,628	v5.0 + v6.0	Soft voting	53.40%	48.25%	84.10%	Best overall

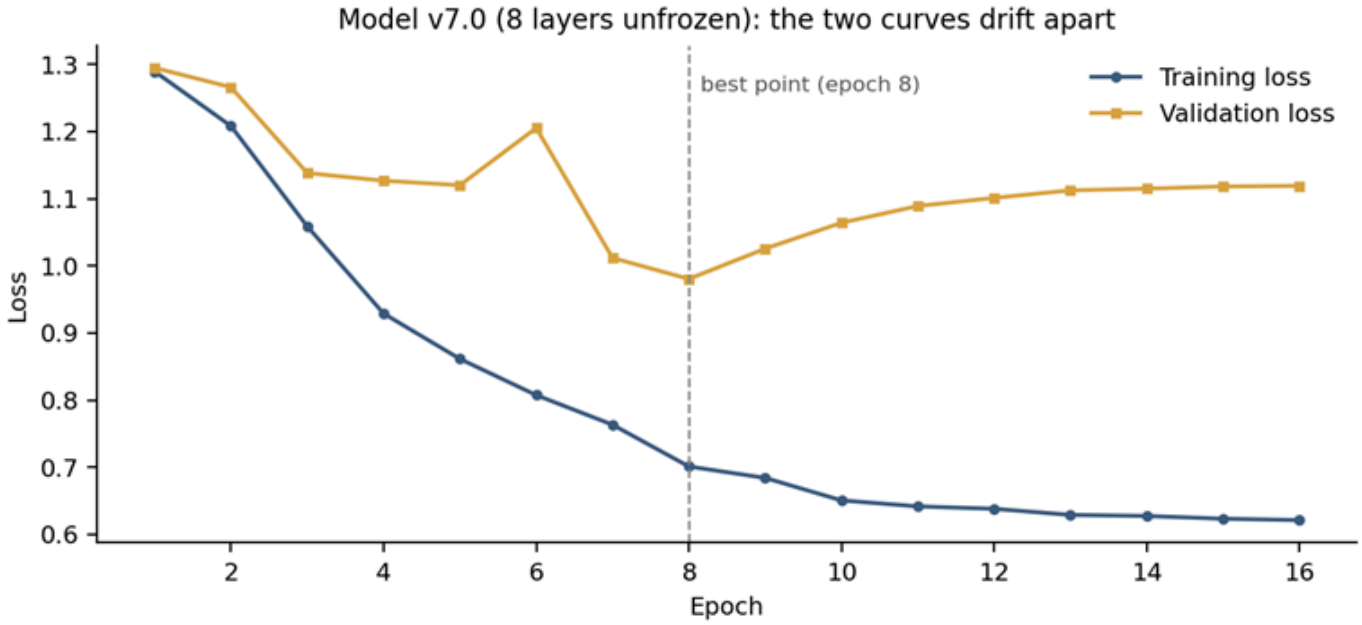


Accuracy and macro F1 across model versions.

Step 7: Fine tuning the model.

From epoch 8 to 16 the training loss went down from 0.70 to 0.62, so the model was clearly still learning something. But validation loss went up from 0.98 to 1.12, and accuracy dropped from 40.35% to 35.55%. The explanation we believe in is this. In speech transformers, the middle layers (roughly 12 to 19) hold a lot of information about who is speaking: voice pitch, vocal tract shape, timbre. The top layers (20 to 23) hold more abstract information such as phonetic content and intonation. Our corpus comes from television, where the same newsreaders and interviewees appear again and again. When we unfroze layers 16 to 23, the model gained access to voice-identity information, and the easiest way to lower the training loss was to recognise people, not dialects. It memorised the anchors' voices. With only layers 20 to 23 unfrozen, the voice-identity layers stay locked, so the only route to a better score is to learn dialect

features that carry across speakers. That is why four layers generalise best.



Training loss keeps falling while validation loss rises after epoch 8.

Step 8: Results;

The dual-engine ensemble averages the probabilities of v5.0 and v6.0 (weights 0.55 and 0.45) and reaches 53.40% accuracy, 48.25% macro F1 and 84.10% Top-2 accuracy. For context, random guessing on eight classes would be correct 12.5% of the time, so this is more than four times better than chance on a task that is hard even for people.

Web dashboard and desktop client : A Gradio web page lets people record from a microphone or upload files and see results immediately. We also built a native Java 17 desktop application with Swing. It has a drag-and-drop zone, a live 16 kHz microphone recorder, an animated bar chart of all eight probabilities (the winning dialect in blue,

the others in grey), and a card that explains the geography and character of the predicted dialect. HTTP calls run in background threads, so the window never freezes while the model is working. A command-line client prints the result as text bars and JSON, and shell scripts compile and launch everything with one command.

Explaining each answer ; Under the “Analyse” button, a rationale card lists the sound features that typify the predicted dialect (for example, elevated pitch and long open final vowels for Bardez, or crisp dental stops for Sashti Hindu), shows the percentage margin between the first and second choices, and tells the user whether both models agreed. This makes the result easier to question and to teach with.

Main Features in summary:

• Drag-and-drop or browse for WAV, MP3, M4A, FLAC, AAC and OGG files.

• Live recording at 16 kHz mono from the microphone, with timer and playback.

• Automatic cleaning (80 Hz high-pass, 7,600 Hz low-pass, noise reduction, leading silence removal) of every input.

• Handles long recordings by 6-second sliding windows and averaging.

• Choice of engine: v5.0, v6.0, dual ensemble, or tri-engine ensemble (v5.0 + v6.0 + v7.0 with weights 35/35/30).

• Animated bar chart of all eight dialect probabilities.

• Agreement badge: “High-confidence consensus” or “Ambiguous regional boundary”.

• Prediction rationale card with typical sound features, margin over the runner-up and model agreement.

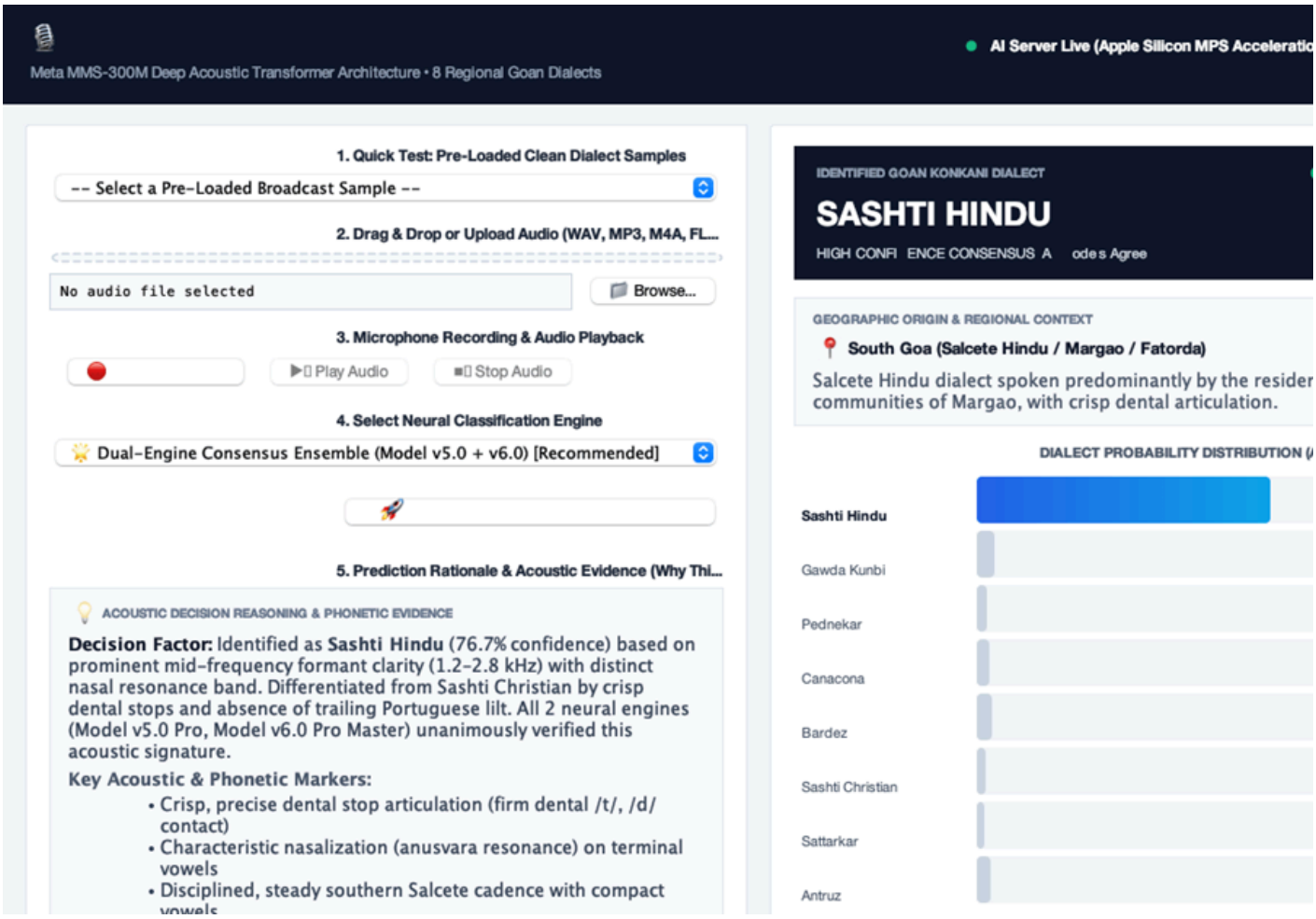
• Geography and culture card for the predicted dialect.

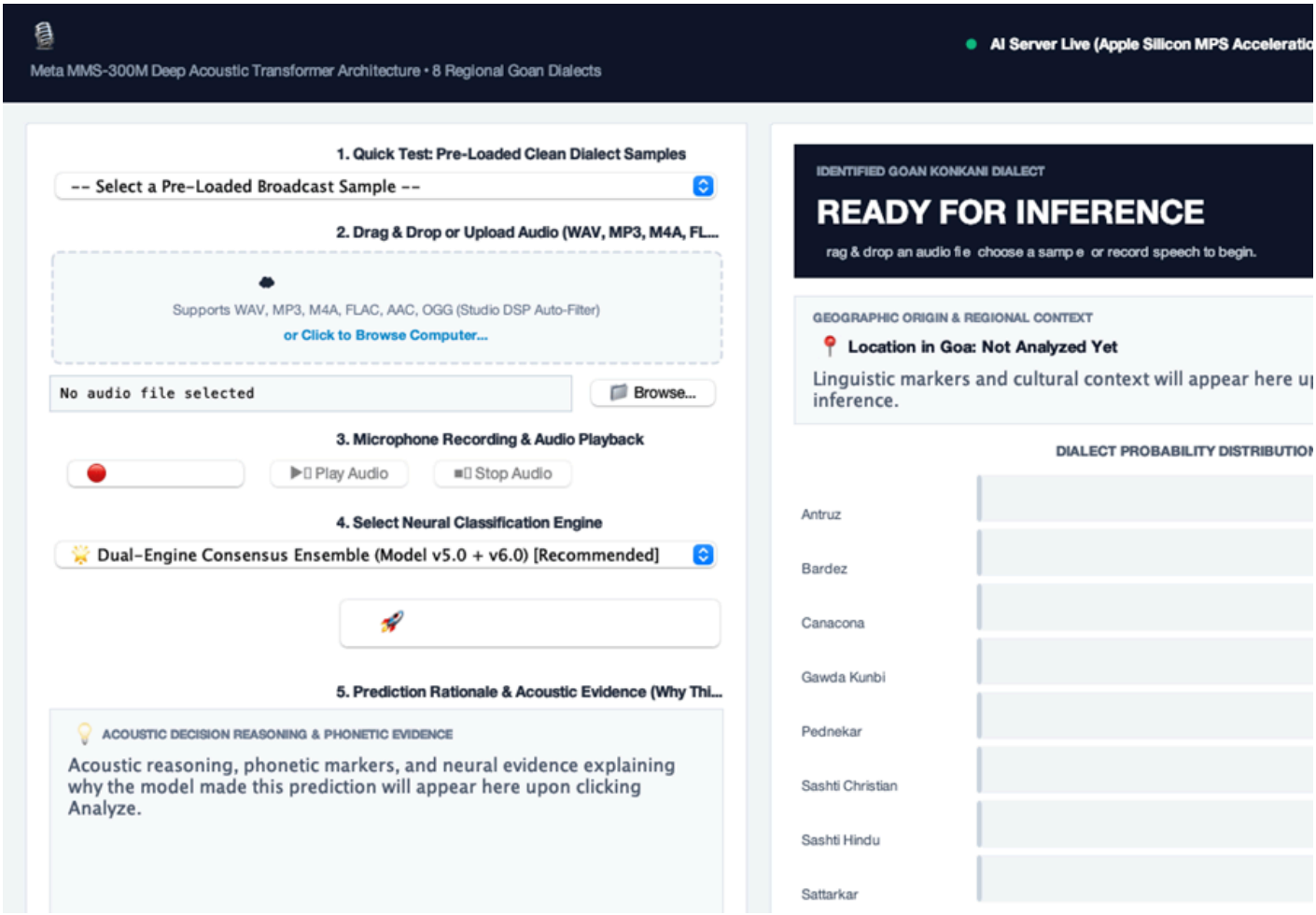
• Command-line client and shell scripts for batch use

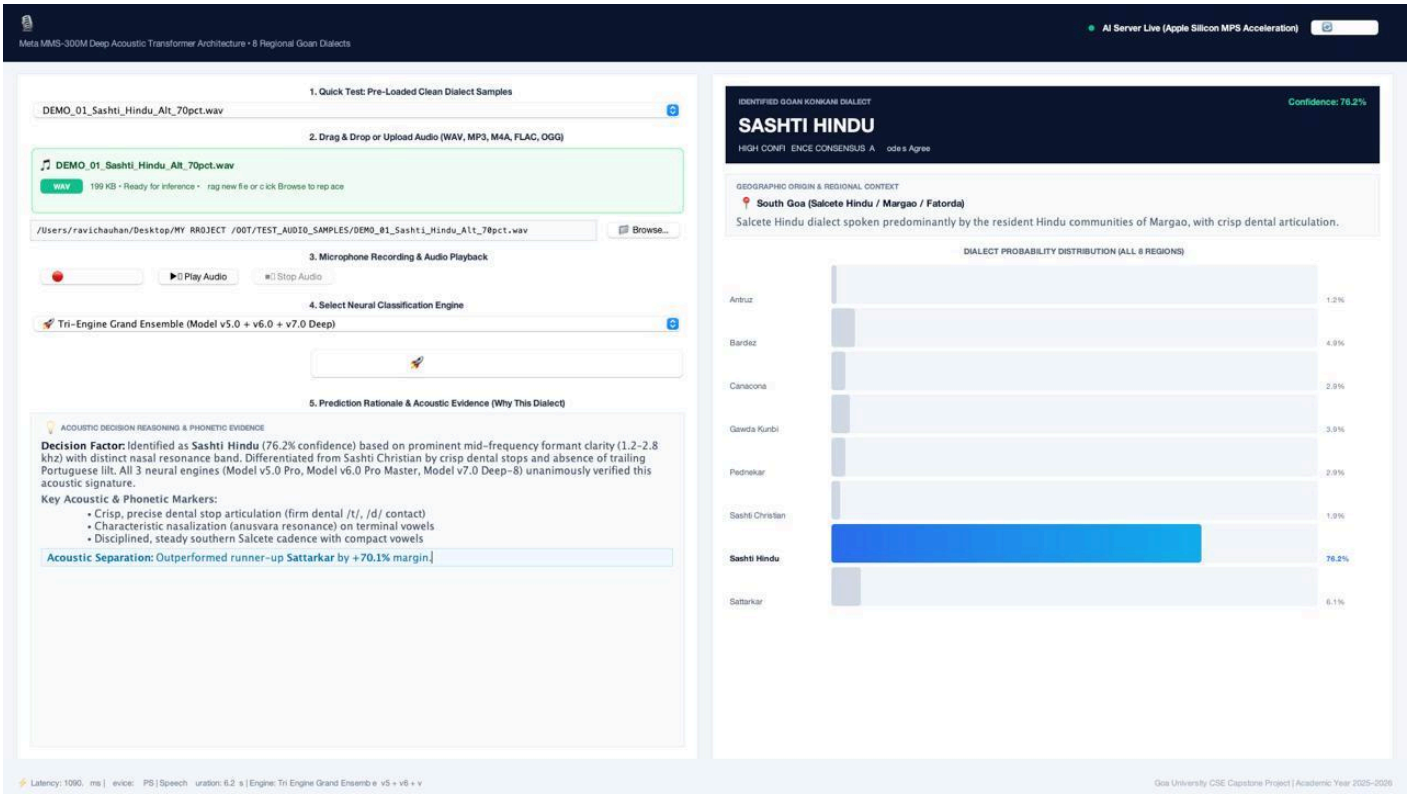
Testing and verification

We tested the system at three levels. At the unit level, each audio filter was checked on synthetic hum and noise recordings to confirm the intended attenuation (for instance, more than 24 dB at 8 kHz for the low-pass stage). At the model level, results were measured on the 1,145 held-out clips with accuracy, precision, recall, macro F1 and Top-2 accuracy. At the system level, we ran the complete flow of recording, upload, server call and result display on sample files, for example a Sashti Hindu clip from Margao that the system identified correctly with 76.7% confidence. We also checked that empty files, very short clips and corrupted audio gave clear error messages instead of crashes.

LIVE SCREENSHOTS




Meta MMS-300M Deep Acoustic Transformer Architecture • 8 Regional Goan Dialects
AI Server Live (Apple Silicon MPS Acceleratio
1. Quick Test: Pre-Loaded Clean Dialect Samples
-- Select a Pre-Loaded Broadcast Sample --
2. Drag & Drop or Upload Audio (WAV, MP3, M4A, FL...
Supports WAV, MP3, M4A, FLAC, AAC, OGG (Studio DSP Auto-Filter)
or Click to Browse Computer...
No audio file selected
Browse...
3. Microphone Recording & Audio Playback
Play Audio
Stop Audio
4. Select Neural Classification Engine
Dual-Engine Consensus Ensemble (Model v5.0 + v6.0) [Recommended]
5. Prediction Rationale & Acoustic Evidence (Why Thi...
ACOUSTIC DECISION REASONING & PHONETIC EVIDENCE
Acoustic reasoning, phonetic markers, and neural evidence explaining why the model made this prediction will appear here upon clicking Analyze.
IDENTIFIED GOAN KONKANI DIALECT
READY FOR INFERENCE
rag & drop an audio fie choose a samp e or record speech to begin.
GEOGRAPHIC ORIGIN & REGIONAL CONTEXT
Location in Goa: Not Analyzed Yet
Linguistic markers and cultural context will appear here u
inference.
DIALECT PROBABILITY DISTRIBUTION
Antruz
Bardez
Canacona
Gawda Kunbi
Pednekar
Sashti Christian
Sashti Hindu
Sattarkar


Meta MMS-300M Deep Acoustic Transformer Architecture • 8 Regional Goan Dialects
AI Server Live (Apple Silicon MPS Acceleration)
1. Quick Test: Pre-Loaded Clean Dialect Samples
DEMO_01_Sashti_Hindu_Alt_70pct.wav
2. Drag & Drop or Upload Audio (WAV, MP3, M4A, FLAC, OGG)
DEMO_01_Sashti_Hindu_Alt_70pct.wav
WAV
199 KB • Ready for inference •
/Users/ravichauhan/Desktop/MY RROJECT /OOT/TEST_AUDIO_SAMPLES/DEMO_01_Sashti_Hindu_Alt_70pct.wav
Browse...
3. Microphone Recording & Audio Playback
Play Audio
Stop Audio
4. Select Neural Classification Engine
Tri-Engine Grand Ensemble (Model v5.0 + v6.0 + v7.0 Deep)
5. Prediction Rationale & Acoustic Evidence (Why This Dialect)
ACOUSTIC DECISION REASONING & PHONETIC EVIDENCE
Decision Factor: Identified as Sashti Hindu (76.2% confidence) based on prominent mid-frequency formant clarity (1.2-2.8 khz) with distinct nasal resonance band. Differentiated from Sashti Christian by crisp dental stops and absence of trailing Portuguese lilt. All 3 neural engines (Model v5.0 Pro, Model v6.0 Pro Master, Model v7.0 Deep-8) unanimously verified this acoustic signature.
Key Acoustic & Phonetic Markers:
• Crisp, precise dental stop articulation (firm dental /t/, /d/ contact)
• Characteristic nasalization (anusvara resonance) on terminal vowels
• Disciplined, steady southern Salcete cadence with compact vowels
Acoustic Separation: Outperformed runner-up Sattarkar by +70.1% margin.
IDENTIFIED GOAN KONKANI DIALECT
Confidence: 76.2%
SASHTI HINDU
HIGH CONFI ENCE CONSENSUS A ode s Agree
GEOGRAPHIC ORIGIN & REGIONAL CONTEXT
South Goa (Salcete Hindu / Margao / Fatorda)
Salcete Hindu dialect spoken predominantly by the resident Hindu communities of Margao, with crisp dental articulation.
DIALECT PROBABILITY DISTRIBUTION (ALL 8 REGIONS)
Antruz
1.2%
Bardez
4.9%
Canacona
2.9%
Gawda Kunbi
3.9%
Pednekar
2.9%
Sashti Christian
1.9%
Sashti Hindu
76.2%
Sattarkar
6.1%
Latency: 1090. ms | evice: PS | Speech uration: 6.2 s | Engine: Tri Engine Grand Ensemb e v5 + v6 + v
Goa University CSE Capstone Project | Academic Year 2025–2026

LIMITATION OF PROPOSED SYSTEM

- **Accuracy is useful but not yet high:** The best result is 53.40% accuracy over eight classes. That is far above chance (12.5%) and the Top-2 score of 84.10% is encouraging, but the system will still pick the wrong dialect nearly half the time. It should be used as an assistant that suggests, not as a judge that decides.
- **Dialects are a continuum:** Real speech does not respect taluka boundaries. A person from a border village naturally sounds like two dialects at once, and anyone who has moved between regions may carry traces of several. Forcing such speakers into one of eight classes is unavoidable in a classification system, but it will always be an approximation.
- **Labels are inferred, not verified** : We labelled clips by matching place names in their metadata. A clip tagged “Bardez” because the programme was recorded in Mapusa may feature a speaker who grew up elsewhere. This label noise puts a ceiling on achievable accuracy, and ideally every clip would be checked by a native speaker.
- **Television speech is not everyday speech:** Our training audio is broadcast speech, which tends to be clearer, more careful and closer to a standard than casual conversation. Field recordings in a fish market, on a bus, or during monsoon rain will have more noise, more overlapping speech and more code-switching into English and Hindi. Performance on such audio is likely to be lower.

- **Repeated speakers:** The corpus has recurring newsreaders and guests. We kept programmes and speakers on one side of the train/test split to avoid leakage, and the 8-layer experiment shows how easily a model can learn voices instead of dialects. Even so, we cannot guarantee the models have not picked up some speaker or channel effects.
- **Uneven data:** Even after balancing, the largest class (Antruz) is still 4.7 times bigger than the smallest (Kudali). Small classes have lower F1 scores (Kudali 38.4%, Pednekari 40.7%) than large ones. The test set is also modest in size, with only 69 Kudali and 74 Sattarkar clips, so individual per-class scores carry some statistical uncertainty .
- The system needs at least 1 second, and ideally 5 to 15 seconds, of continuous speech; single words are not enough.

FUTURE SCOPE

- **Short term:** put it in people's pockets (months 1–6) Compress the models to 8-bit precision (INT8 quantisation) and export them with ONNX Runtime and Apple CoreML. The aim is a small offline model, under about 80 MB, that runs on ordinary Android and iOS phones. That would let teachers, students and field researchers use the tool anywhere, without internet.
- **Medium term:** talking assistants that speak your dialect (months 6–12) Combine the dialect probabilities with Indic language models such as Sarvam AI's or Indic Llama-3 variants, so a voice assistant can recognise how a user speaks and reply in a matching register. This is a small step technically but a large one culturally.
- **Meetings with many speakers:** (months 12–18) Add speaker diarisation (for example with PyAnnote) so that panchayat meetings, assembly debates or interviews with several people can be split by speaker and each turn labelled with its dialect.
- **A community-built corpus** :(months 18–24) The largest improvement would come from better data. We propose a mobile web app that lets villagers donate short recordings, tagged with their village, age group and the language they grew up with. Targeting all 189 panchayats and reaching 1,000+ hours would also make it possible to study micro-dialects of smaller communities .

Other research ideas

• Soft labels: instead of one label per clip, give the model a mixture such as “70% Bardez, 30% Pednekari”, to match the continuum.

• Speaker-adversarial training: penalise the model whenever its features reveal the speaker’s identity, to reduce voice memorisation even when more layers are trained.

• Other regions: extend to Konkani in coastal Karnataka, Maharashtra and Kerala, and to related languages like Marathi dialects.

• Learning from the dialect map: let the model use geographic knowledge, for instance by predicting a location rather than a class.

• Teaching and heritage tools: build language-learning games and archive search tools that use the dialect labels.

CONCLUSION

This project began with a simple observation: Goans can hear where someone is from, but computers cannot. We set out to change that for Konkani, a language that matters deeply to the people who speak it and that technology has largely ignored. We showed that the usual speech-to-text-then-classify approach is a poor fit for Konkani because of its multiple scripts, unstable spelling and, above all, because writing speech down removes the pitch, rhythm and nasality that define dialects.

Instead, we built Goan Konkani Dialect Identification Using Speech Audio, which works directly on sound. To do this we assembled the first labelled, multi-channel dialect dataset of its kind for Goan Konkani (7,628 clips, about 59.5 hours) from 108.5 hours of broadcast speech, cleaned it with a four-step audio filter, and fine-tuned the MMS-300M speech model with a class-weighted focal loss. Our experiments led to four main findings:

1. Letting the model hear the audio directly works: 51.18% accuracy for a single model and 53.40% for the two-model ensemble, with 84.10% Top-2 accuracy.
2. Mistakes fall between geographic neighbours, so the model has learned something real about how Goan dialects relate to each other.
3. Fine-tuning only the top four layers is best. Going deeper, to eight layers, made the model memorise television anchors' voices and lowered accuracy to 40.35%.
4. Cleaning the audio needs dialect awareness: trimming "silence" at the end of clips removed the very vowels that carry dialect information.

The system runs in about 48 milliseconds per 6-second clip on a laptop, comes with

a web dashboard, a desktop client and a command-line tool, and explains each answer in plain language. There is plenty left to do: more and better-verified data, a better treatment of border speakers, and a phone-sized version. But the main point is made. With a good foundation model, careful data work and a bit of linguistic understanding, it is possible to build technology that listens to the way Goans actually speak. We hope this report is a useful step towards speech tools that respect, and help preserve, the rich variety of Konkani.

BIBLIOGRAPHY

- A. Baevski, H. Zhou, A. Mohamed and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 12449–12460, 2020.
- V. Pratap, A. Tjandra, B. Shi, et al., “Scaling speech technology to 1,000+ languages,” arXiv:2305.13516, Meta AI Research, 2023.
- T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, “Focal loss for dense object detection,” IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988, 2017.
- D. Snyder, D. Garcia-Romero, G. Sell, D. Povey and S. Khudanpur, “X-vectors: Robust DNN embeddings for speaker recognition,” IEEE ICASSP, pp. 5329–5333, 2018.
- B. Desplanques, J. Thienpondt and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification,” Interspeech, pp. 3830–3834, 2020.
- N. Dehak, P. Kenny, R. Dehak, P. Dumouchel and P. Ouellet, “Front-end factor analysis for speaker verification,” IEEE Transactions on Audio, Speech, and Language Processing, vol. 19, no. 4, pp. 788–798, 2011.
- S. M. Katre, The Formation of Konkani, Deccan College Building Centenary and Silver Jubilee Series, Poona: Deccan College Postgraduate and Research Institute, 1966.
- A. M. Ghatage, Konkani of South Kanara, Survey of Marathi Dialects, Bombay: State Board for Literature and Culture, 1971.
- I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” International Conference on Learning Representations (ICLR), 2019.

- D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” Interspeech, pp. 2613–2617, 2019.
- D. A. Reynolds, T. F. Quatieri and R. B. Dunn, “Speaker verification using adapted Gaussian mixture models,” Digital Signal Processing, vol. 10, no. 1–3, pp. 19–41, 2000.
- A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- Reuben CF, “Goan Konkani Speech Corpus,” Hugging Face Datasets (Reubencf/goan-konkani-speech), 2023.
- Shenoi Goembab (V. R. Varde Valaulikar), Konkani Bhashechem Zait, Bombay: Gomantak Press, 1930.
- P. Kakodkar, The Goa Inquisition, Bombay: Academic Press, 1961. Konkani Kaan: Telling Goan Dialects Apart by Sound Chapter 14: Bibliography 56 [16] Central Institute of Indian Languages (CIIL), Linguistic Survey of India: Konkani Language Monograph, Ministry of Education, Government of India, 2012.
- 56A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” International Conference on Machine Learning (ICML), pp. 28492–28518, 2023.